Panduan Testing Aplikasi Web

Dari Dasar hingga Produksi: Sebelum Deploy, Saat Deploy, dan Setelah Deploy

Praktik terbaik 2026 • contoh nyata memakai Django (proyek SPBD-PLP)

Materi belajar • Bahasa Indonesia

Daftar Isi

1. Mengapa testing, dan peta besarnya
2. Dasar-dasar: jenis test dan strategi
3. Persiapan: lingkungan, data, dan aturan main
4. TAHAP 1 - Sebelum deploy (di laptop dan CI)
5. TAHAP 2 - Saat deploy (gerbang rilis dan strategi aman)
6. TAHAP 3 - Setelah deploy (produksi)
7. Pipeline CI/CD contoh lengkap
8. Penerapan pada proyek Django (SPBD-PLP)
9. Daftar alat 2026
10. Checklist go / no-go
11. Kesalahan umum dan jalur belajar

> **Cara memakai materi ini.** Baca bab 1-3 sekali sebagai fondasi. Bab 4-6 adalah inti: urutan kerja yang dijalankan setiap rilis. Bab 7-10 adalah bahan praktik: salin, sesuaikan, jalankan.

1. Mengapa Testing, dan Peta Besarnya

Testing bukan untuk membuktikan aplikasi *tanpa bug*; mustahil. Tujuannya menurunkan **risiko** sampai level yang bisa diterima sebelum pengguna merasakannya, dan mendeteksi masalah secepat mungkin sesudahnya. Biaya memperbaiki bug naik berkali lipat semakin jauh ia lolos: ditemukan saat menulis kode (murah), saat review (sedang), di produksi (mahal, dan merusak kepercayaan).

Tiga prinsip utama 2026

- **Shift-left**: pindahkan pengecekan sedekat mungkin ke saat kode ditulis (lint, tipe, secret scan, unit test dijalankan otomatis tiap commit).
- **Shift-right**: testing tidak berhenti di deploy. Observability, monitoring sintetis, dan canary adalah bagian dari testing.
- **Otomatisasi + gerbang (gate)**: semua yang bisa diotomatisasi masuk pipeline CI/CD; rilis hanya jalan bila gerbang hijau.

Peta besar: tiga tahap

Tahap	Tujuan	Contoh kegiatan
1. Sebelum deploy	Mencegah cacat masuk ke rilis	Static analysis, unit/integrasi/E2E, keamanan, performa, aksesibilitas, uji migrasi
2. Saat deploy	Merilis dengan risiko terkecil	Gerbang CI, staging, smoke test, canary / blue-green, feature flag, rencana rollback
3. Setelah deploy	Memastikan nyata berjalan sehat	Smoke produksi, monitoring sintetis, error tracking, metrik, uji pemulihan backup, pemindaian berkala

2. Dasar-dasar: Jenis Test dan Strategi

2.1 Piramida dan 'trophy' testing

Piramida klasik: banyak **unit test** (cepat, murah), lebih sedikit **integration test**, sedikit **E2E** (lambat, rapuh tapi paling mirip pengguna). Untuk aplikasi web modern banyak tim memakai bentuk *trophy*: fondasi static analysis, inti terbesar di integration test, ditutup E2E untuk alur kritis saja.

```
          /\        E2E / UI (sedikit, alur kritis)
         /  \
        /----\      Integrasi / API / otorisasi (porsi terbesar)
       /------\
      /--------\    Unit test (logika murni, cepat)
     /----------\
    ==============  Static analysis: lint, tipe, SAST, secret scan
```

2.2 Jenis test dan kapan dipakai

Jenis	Menjawab pertanyaan	Contoh alat
Static analysis	Apakah kode bermasalah tanpa dijalankan?	ruff, mypy/pyright, ESLint, bandit, semgrep
Unit test	Apakah satu fungsi/kelas benar?	pytest, Django TestCase, Vitest/Jest
Integrasi	Apakah komponen (view+DB+form) bekerja bersama?	Django test client, pytest-django, Testcontainers

Jenis	Menjawab pertanyaan	Contoh alat
Contract / API	Apakah API sesuai kesepakatan skema?	schemathesis, Pact, OpenAPI validator
E2E / UI	Apakah alur pengguna nyata berhasil?	Playwright (utama 2026), Cypress
Regresi visual	Apakah tampilan berubah tak sengaja?	Playwright screenshots, Percy, Chromatic
Keamanan	Apakah bisa diretas / disalahgunakan?	bandit, pip-audit, ZAP, gitleaks, Trivy
Performa / beban	Kuat berapa pengguna? Seberapa cepat?	k6, Locust, Lighthouse CI
Aksesibilitas	Bisa dipakai semua orang?	axe-core, Lighthouse, uji keyboard + screen reader
Kompatibilitas	Jalan di semua peramban/ukuran layar?	Playwright (Chromium, Firefox, WebKit), BrowserStack
Smoke / sanity	Apakah hal paling vital hidup?	Skrip kecil + curl/Playwright
Exploratory / UAT	Apakah sesuai kebutuhan nyata pengguna?	Manusia: sesi terbimbing + checklist

2.3 Pola penulisan test yang baik

- **Arrange - Act - Assert**: siapkan data, lakukan aksi, periksa hasil. Satu test = satu alasan gagal.
- **Deterministik**: tanpa ketergantungan waktu, urutan, atau internet. Bekukan waktu, isi data sendiri (fixture/factory).
- **Independen**: test boleh dijalankan sendiri-sendiri dan acak urutan.
- **Uji perilaku, bukan implementasi**: test tidak ikut patah saat refactor yang benar.
- **Nama jelas**: `test_koordinador_suku_tidak_bisa_lihat_membru_suku_lain`.
- **Uji jalur gagal**: input kosong, terlalu panjang, karakter aneh, tidak berhak, dobel-klik, koneksi putus.

> **Flaky test** (kadang lulus, kadang gagal) merusak kepercayaan pada seluruh suite. Aturan: perbaiki akar masalah (wait eksplisit, data terisolasi), jangan hanya dijalankan ulang atau dinonaktifkan.

2.4 Ukuran kualitas (secukupnya)

- **Coverage** berguna sebagai radar lubang, bukan target angka. Pantau branch coverage pada modul kritis (otorisasi, keuangan, import data).
- **Mutation testing** (mutmut, Stryker) menguji kualitas test itu sendiri: jika kode dirusak sedikit, apakah test gagal?
- **Waktu suite**: jaga PR-gate di bawah ~10 menit; pindahkan yang lambat ke jadwal malam.

3. Persiapan: Lingkungan, Data, dan Aturan Main

3.1 Lingkungan (environment)

Lingkungan	Fungsi	Aturan
Lokal / dev	Menulis dan menguji cepat	DEBUG boleh hidup; data palsu; jangan pernah pakai database produksi

Lingkungan	Fungsi	Aturan
CI	Pemeriksaan otomatis tiap push/PR	Bersih tiap run, mudah direproduksi, tanpa rahasia produksi
Staging	Replika produksi untuk uji akhir	Konfigurasi, versi, dan topologi semirip mungkin dengan produksi; data anonim
Produksi	Pengguna sungguhan	DEBUG mati, rahasia di secret manager, akses minimum, monitoring aktif

3.2 Data uji

- Gunakan **factory/fixture** kecil dan jelas, bukan dump produksi.
- Jika perlu data mirip produksi: **anonimkan** (hapus nama, nomor telepon, email, nomor identitas). Data pribadi anggota partai termasuk sensitif.
- Siapkan **seed** peran lengkap (admin, presidente, koordenador, anggota biasa) agar uji otorisasi mudah.

3.3 Definition of Done & aturan cabang

- Fitur dianggap selesai bila: ada test, lulus CI, ada review, dokumentasi/migrasi tercatat.
- Branch utama dilindungi: wajib PR, wajib CI hijau, tidak boleh push langsung.
- Setiap bug produksi berakhir dengan **test regresi** yang gagal dulu lalu lulus setelah diperbaiki.

4. TAHAP 1 - Sebelum Deploy

Urutan di bawah dirancang dari yang **tercepat dan termurah** ke yang paling lambat, sehingga kegagalan terdeteksi secepat mungkin.

4.1 Langkah 1: Pemeriksaan statis (hitungan detik)

- **Format & lint**: ruff / black / ESLint. Satu gaya kode, tanpa debat.
- **Tipe**: mypy/pyright (Python), TypeScript strict. Menangkap kelas bug besar sejak awal.
- **SAST** (analisis keamanan kode): bandit dan semgrep untuk Python/Django; temukan `eval`, SQL mentah, hash lemah, `mark_safe` berbahaya.
- **Secret scan**: gitleaks / trufflehog / GitHub secret scanning. Kunci API, password, `.env` tidak boleh masuk repo.
- **Dependensi**: pip-audit / npm audit / Dependabot / Renovate untuk kerentanan; kunci versi (lockfile).
- **SBOM** (daftar komponen perangkat lunak): hasilkan dengan CycloneDX/Syft; makin sering diminta regulasi dan klien.
- **IaC & container** (jika ada): Trivy / hadolint untuk Dockerfile dan image.

```
ruff check . && ruff format --check .
mypy .
bandit -r . -x tests -ll
gitleaks detect --no-banner
pip-audit -r requirements.txt
```

4.2 Langkah 2: Unit test

Uji logika murni: validator, fungsi hitung umur, normalisasi nama, aturan 'siapa boleh melakukan apa'. Harus selesai dalam hitungan detik-menit.

```
# contoh pytest / Django
def test_nomor_telepon_dinormalkan():
    assert normalkan("+670 7712 3456") == "67077123456"

def test_csv_aman_menetralkan_formula():
    assert csv_aman("=HYPERLINK('x')").startswith("'")
```

4.3 Langkah 3: Integrasi (view + form + database)

- Gunakan **database sungguhan** (SQLite/PostgreSQL/MySQL sesuai produksi) - jangan mock yang membuat test bohong.
- Uji alur: login, buat data, validasi form, simpan, tampil di daftar, ekspor.
- **Uji migrasi**: jalankan `migrate` dari nol *dan* dari salinan data lama; pastikan `makemigrations --check` bersih.
- **Uji impor data**: berkas Excel/CSV berantakan (kolom kosong, duplikat, karakter aneh) tidak boleh merusak data.

```
python manage.py makemigrations --check --dry-run
python manage.py test --parallel
```

4.4 Langkah 4: Uji otorisasi (paling sering terlewat)

Kebanyakan kebocoran data nyata bukan karena peretas canggih, melainkan **pengguna yang berhak sedikit membaca data milik orang lain** (Broken Access Control - peringkat teratas OWASP). Untuk sistem berperan seperti SPBD-PLP:

- Buat **matriks peran x URL**: untuk setiap halaman dan setiap peran, apa yang diharapkan (200 / 403 / 302 ke login)?
- Uji **lingkup wilayah**: Koordenador Suku A tidak boleh melihat/mengubah data Suku B, termasuk lewat ID yang ditebak di URL (IDOR).
- Uji **anonim**: semua halaman internal harus mengalihkan ke login.
- Uji **metode HTTP**: aksi yang mengubah data hanya lewat POST + token CSRF.

> Di SPBD-PLP ini sudah diotomatisasi: `config/tests_audit.py` menjelajahi setiap URL dengan setiap peran dan mencocokkan hasilnya.

4.5 Langkah 5: E2E dengan Playwright

Hanya alur kritis (5-15 skenario): login + 2FA, tambah anggota, persetujuan, ekspor laporan, ganti password. Playwright menjalankan Chromium, Firefox, dan WebKit dengan satu kode.

```
# tests/e2e/test_login.py  (pytest-playwright)
def test_login_dan_lihat_dashboard(page, live_server):
    page.goto(f"{live_server.url}/login/")
    page.get_by_label("Username").fill("presidente")
    page.get_by_label("Password").fill("Rahasia-Uji-123")
    page.get_by_role("button", name="Tama").click()
    page.wait_for_url("**/")
    assert page.get_by_role("heading", name="Dashboard").is_visible()
```

- Pilih elemen dengan **role / label / teks** (seperti pengguna), bukan CSS yang rapuh.
- Andalkan **auto-wait** Playwright, hindari `sleep`.
- Simpan **trace, video, screenshot** saat gagal sebagai artefak CI.
- Uji **responsif**: jalankan skenario inti pada viewport ponsel (375px), tablet (768px), desktop (1280px).

4.6 Langkah 6: Uji keamanan aplikasi

a) Kerangka acuan

- **OWASP Top 10** (edisi terbaru): Broken Access Control, Cryptographic Failures, Injection, Insecure Design, Security Misconfiguration, Vulnerable Components, Authentication Failures, Integrity Failures, Logging & Alerting Failures, SSRF/handling kondisi luar biasa.
- **OWASP ASVS** sebagai daftar periksa terperinci (pilih level 1 untuk dasar, level 2 untuk data sensitif).
- **OWASP Cheat Sheet Series** untuk panduan praktis tiap topik.

b) Yang diuji secara konkret

Area	Cara uji	Hasil yang benar
Login & sesi	Salah password berulang; cookie tanpa flag; sesi setelah logout	Terkunci sementara; cookie Secure+HttpOnly+SameSite; sesi mati

Area	Cara uji	Hasil yang benar
2FA	Kode salah/kedaluwarsa/dipakai ulang	Ditolak; rate limit aktif
XSS	Isi nama dengan `<script>alert(1)</script>`	Tampil sebagai teks, tidak dieksekusi
SQL injection	Input `' OR 1=1 --` di pencarian/filter	Diperlakukan sebagai teks (ORM berparameter)
CSRF	POST tanpa token	403
IDOR	Ganti ID di URL ke milik orang lain	403/404
Upload berkas	Berkas besar, ekstensi palsu, gambar rusak	Ditolak; disimpan privat
Ekspor CSV	Sel diawali `= + - @`	Dinetralkan (CSV injection)
Redirect	Parameter `next=https://jahat.com`	Hanya ke host sendiri
Header	Periksa respons HTTP	HSTS, X-Content-Type-Options, frame protection, Referrer-Policy, CSP
Error	Picu error	Halaman umum, tanpa stack trace

c) Alat otomatis

```
python manage.py check --deploy             # konfigurasi Django aman untuk produksi
bandit -r . -ll                             # SAST
pip-audit                                   # kerentanan dependensi
docker run -t zaproxy/zap-stable zap-baseline.py -t https://staging.contoh.org   # DAST pasif
```

> DAST (ZAP, Burp) **hanya** dijalankan pada sistem milik sendiri atau yang diizinkan tertulis - idealnya staging, bukan produksi dengan data nyata.

4.7 Langkah 7: Performa dan beban

- **Core Web Vitals** (target 'baik'): LCP < 2,5 dtk, INP < 200 ms, CLS < 0,1. Ukur dengan Lighthouse CI di tiap PR.
- **Beban**: simulasikan pengguna serentak (k6 atau Locust) pada halaman berat: daftar anggota besar, dashboard, ekspor.
- **Query**: periksa N+1 query (django-debug-toolbar, `assertNumQueries`); tambahkan index; paginasi daftar besar.
- Jenis uji: *load* (beban normal), *stress* (sampai patah), *spike* (lonjakan tiba-tiba), *soak* (berjam-jam, cari kebocoran memori).

```
// k6: 50 pengguna virtual selama 2 menit
import http from 'k6/http'; import { check } from 'k6';
export const options = { vus: 50, duration: '2m',
  thresholds: { http_req_duration: ['p(95)<800'], http_req_failed: ['rate<0.01'] } };
export default function () {
  const r = http.get('https://staging.contoh.org/login/');
  check(r, { 'status 200': (x) => x.status === 200 });
}
```

4.8 Langkah 8: Aksesibilitas & kompatibilitas

- Otomatis: axe-core (lewat Playwright) + Lighthouse; menangkap ~30-40% masalah.

- Manual: navigasi **hanya dengan keyboard** (Tab, Enter, Esc), fokus terlihat, kontras warna (WCAG 2.2 AA: 4,5:1 untuk teks biasa), label pada setiap input, teks alternatif gambar.
- Uji di ponsel sungguhan minimal satu Android dan satu iOS; periksa tabel lebar, menu samping, dan form pada layar sempit.
- Peramban: versi terbaru Chrome, Firefox, Safari, Edge.

4.9 Langkah 9: Kesiapan operasional

- **Backup & restore**: bukan hanya membuat backup, tetapi *benar-benar memulihkannya* di lingkungan uji dan memastikan data utuh.
- **Konfigurasi**: semua rahasia dari environment/secret manager; `DEBUG=False`; `ALLOWED_HOSTS` benar; HTTPS dipaksa.
- **Log**: ada, terstruktur, tanpa data pribadi dan tanpa password/token.
- **Rencana rollback** tertulis: kembali ke versi sebelumnya, termasuk migrasi database yang bisa dibalik (atau pola expand/contract).
- **Dokumentasi**: README, langkah deploy, kontak darurat.

5. TAHAP 2 - Saat Deploy

5.1 Gerbang rilis (quality gates)

Pipeline CI/CD adalah penjaga pintu. Rilis **hanya** berjalan bila semua gerbang hijau, tanpa pengecualian lisan.

Gerbang	Syarat lulus
Static + keamanan	Lint, tipe, bandit, secret scan, pip-audit tanpa temuan tinggi/kritis
Test otomatis	Unit, integrasi, otorisasi, E2E semua hijau
Migrasi	`makemigrations --check` bersih; migrate sukses pada salinan data
Build	Artefak/image terbangun, dipindai (Trivy), versi bertanda (tag) yang tidak bisa diubah
Persetujuan	Review kode selesai; untuk rilis besar, persetujuan manual pemilik produk

5.2 Urutan deploy yang rapi

1. **Bekukan** versi: buat tag rilis (misal v1.4.0) dan catatan perubahan.
2. **Backup** database produksi sesaat sebelum rilis; catat waktunya.
3. **Deploy ke staging** dengan artefak yang *sama persis* yang akan ke produksi (build sekali, deploy berkali-kali).
4. **Smoke + regresi di staging**, termasuk uji migrasi pada data menyerupai produksi.
5. **Deploy ke produksi** dengan strategi bertahap (5.3).
6. **Smoke test produksi** segera (bab 6.1) dan awasi dasbor 15-30 menit pertama.
7. **Umumkan** hasil; bila ada masalah ikuti rencana rollback tanpa ragu.

5.3 Strategi rilis

Strategi	Cara kerja	Cocok untuk
Rolling	Server diganti satu per satu	Aplikasi stateless banyak instance
Blue-Green	Dua lingkungan identik; pindahkan lalu lintas ke 'hijau' bila sehat; 'biru' siaga untuk rollback instan	Ingin rollback tercepat
Canary	Alihkan 1-5% pengguna ke versi baru, pantau, naikkan bertahap	Perubahan berisiko pada trafik besar
Feature flag	Kode ditayangkan tetapi fitur dimatikan; nyalakan untuk kelompok tertentu	Memisahkan deploy dari rilis fitur
Maintenance window	Hentikan layanan singkat, migrasi, nyalakan lagi	Aplikasi kecil/menengah (seperti SPBD-PLP awal)

5.4 Migrasi database aman

- Pola **expand / contract**: (1) tambah kolom baru kompatibel, (2) deploy kode yang menulis ke dua tempat, (3) isi ulang data, (4) pindah baca, (5) baru hapus kolom lama di rilis berikutnya. Versi lama dan baru bisa hidup bersamaan.

- Hindari migrasi yang mengunci tabel besar saat jam sibuk; uji durasinya di staging.
- Selalu ada **backup** dan jalur balik. Rollback kode tidak otomatis membalik data.

6. TAHAP 3 - Setelah Deploy

Deploy berhasil bukan berarti *selesai*. Produksi punya data, trafik, dan perilaku yang tidak ada di staging. Tahap ini disebut juga **shift-right**.

6.1 Smoke test produksi (menit pertama)

Daftar pendek tindakan **baca-saja / aman** yang membuktikan sistem hidup. Jangan membuat data palsu di produksi kecuali dengan akun uji yang jelas ditandai.

- Halaman login terbuka (200) lewat HTTPS, sertifikat valid.
- Login dengan akun uji khusus berhasil; dashboard tampil; satu halaman daftar terbuka.
- Koneksi database sehat (endpoint `/healthz` mengecek DB, cache, penyimpanan).
- Berkas statis (CSS/JS/logo) termuat; tidak ada galat di konsol peramban.
- Tidak ada lonjakan error 5xx di log.

```
curl -fsS -o /dev/null -w "%{http_code} %{time_total}s\n" https://contoh.org/login/
curl -fsS https://contoh.org/healthz
```

6.2 Observability: tiga pilar

Pilar	Isi	Alat umum
Log	Peristiwa terstruktur (JSON): siapa, apa, kapan, hasil. Tanpa password/token/data pribadi	Loki/ELK, CloudWatch, journald
Metrik	Angka dari waktu ke waktu: request/detik, latensi p95, error rate, CPU, memori, antrean	Prometheus + Grafana, Datadog
Trace	Jejak satu permintaan lintas komponen; temukan bagian lambat	OpenTelemetry + Tempo/Jaeger

- **Error tracking**: Sentry (atau setara) menangkap exception beserta konteks dan merilis per versi, sehingga terlihat 'rilis 1.4.0 memunculkan galat baru'.
- **Pantau empat sinyal emas**: latensi, trafik, error, saturasi (kejenuhan sumber daya).
- Tandai **setiap deploy** di grafik agar korelasi sebab-akibat jelas.

6.3 Monitoring sintetis & uptime

- Robot dari luar mengakses alur penting tiap 1-5 menit (UptimeRobot, Better Stack, Grafana Synthetic, atau skrip Playwright terjadwal).
- Alert ke kanal yang benar-benar dibaca (WhatsApp/Telegram/email/pager) dengan **ambang jelas** dan siapa yang bertugas.
- Hindari alert berisik; alert yang terlalu sering akan diabaikan.

6.4 Pengalaman pengguna nyata (RUM)

- Ukur Core Web Vitals dari peramban pengguna sungguhan, bukan hanya laboratorium.
- Kumpulkan umpan balik: tombol lapor masalah, saluran dukungan; tinjau tiap minggu.
- Periksa segmen: perangkat murah, koneksi lambat, wilayah terpencil - sering terjadi pada pengguna lapangan.

6.5 Keamanan berkelanjutan

- **Header & TLS**: periksa berkala dengan securityheaders.com dan SSL Labs; targetkan nilai A.
- **Dependensi**: Dependabot/Renovate membuka PR otomatis; terapkan patch keamanan dalam hitungan hari.
- **Pemindaian terjadwal**: ZAP baseline mingguan di staging; pemindaian image mingguan.
- **Tinjau log login**: percobaan gagal beruntun, login jam tak wajar, akun tidak dipakai - nonaktifkan.
- **Rotasi rahasia** berkala dan segera setelah ada kecurigaan bocor.
- **Uji penetrasi** oleh pihak independen minimal setahun sekali atau setelah perubahan besar - apalagi untuk data pribadi anggota dalam jumlah besar.
- **Kebijakan pengungkapan**: berkas `security.txt` / SECURITY.md agar peneliti tahu ke mana melapor.

6.6 Ketahanan: backup dan pemulihan

- Aturan **3-2-1**: 3 salinan, 2 media berbeda, 1 di luar lokasi.
- **Latihan pemulihan** terjadwal (misal tiap kuartal): restore ke lingkungan uji, hitung waktu (RTO) dan data yang hilang (RPO), dokumentasikan.
- Enkripsi backup; batasi siapa yang boleh mengaksesnya.

6.7 Manajemen insiden dan post-mortem

Fase	Tindakan
Deteksi	Alert / laporan pengguna. Tetapkan tingkat keparahan (SEV1 layanan mati, SEV2 fitur utama rusak, SEV3 minor)
Tanggap	Satu **incident commander**; stabilkan dulu (rollback / matikan feature flag) baru cari penyebab
Komunikasi	Kabari pengguna/pimpinan dengan jujur dan berkala; bila ada kebocoran data pribadi, ikuti kewajiban pelaporan yang berlaku
Pemulihan	Verifikasi dengan smoke test; pantau lebih ketat beberapa jam
Pembelajaran	Post-mortem **tanpa menyalahkan orang**: kronologi, akar masalah, apa yang menahan deteksi, tindakan perbaikan berpemilik dan bertenggat; **tambah test regresi**

6.8 Peninjauan pasca-rilis

- 24 jam: tinjau error baru, performa, keluhan.
- 1 minggu: apakah metrik bisnis/adopsi sesuai harapan? Ada query lambat baru?

- Bersihkan: hapus feature flag yang sudah permanen, hapus kolom/jalur lama (tahap contract), perbarui dokumentasi.

7. Pipeline CI/CD Contoh Lengkap

Contoh GitHub Actions untuk proyek Django. Susunannya: pemeriksaan cepat dulu, test paralel, lalu E2E dan pemindaian, terakhir deploy bersyarat.

```
name: ci
on:
  pull_request:
  push: { branches: [main] }

jobs:
  cepat:                        # < 2 menit
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12", cache: pip }
      - run: pip install -r requirements.txt ruff bandit pip-audit
      - run: ruff check . && ruff format --check .
      - run: bandit -r . -x tests -ll
      - run: pip-audit -r requirements.txt
      - uses: gitleaks/gitleaks-action@v2

  test:
    needs: cepat
    runs-on: ubuntu-latest
    env: { DJANGO_DEBUG: "True", SECRET_KEY: "kunci-hanya-untuk-ci" }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12", cache: pip }
      - run: pip install -r requirements.txt coverage
      - run: python manage.py makemigrations --check --dry-run
      - run: python manage.py check
      - run: coverage run manage.py test --parallel && coverage report --fail-under=80

  e2e:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pip install -r requirements.txt pytest-playwright && playwright install --with-deps
      - run: pytest tests/e2e --tracing=retain-on-failure
      - uses: actions/upload-artifact@v4
        if: failure()
        with: { name: playwright-trace, path: test-results/ }

  deploy-staging:
    needs: [test, e2e]
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    environment: staging
    steps:
      - run: echo "deploy artefak bertag ke staging, lalu smoke test"
```

> Praktik amankan CI: beri token izin minimum (`permissions:`), kunci versi action dengan SHA bila memungkinkan, jangan jalankan secret pada PR dari fork, dan pisahkan secret staging dari produksi.

8. Penerapan pada Proyek Django (SPBD-PLP)

Ringkasan bagaimana bab-bab di atas dipetakan ke proyek SPBD-PLP yang sudah ada di repositori ini.

Kebutuhan	Sudah ada di SPBD-PLP	Langkah lanjut yang disarankan
Unit & integrasi	230 test (`python manage.py test`)	Tambah coverage report dan batas minimum
Otorisasi	`config/tests_audit.py`: URL x peran	Tambah uji IDOR untuk setiap ID baru
Login aman	Lockout, log login, TOTP 2FA, ganti password wajib	Enkripsi secret 2FA; tampilkan QR
SAST & dependensi	bandit + pip-audit di CI	Tambah semgrep dan gitleaks
Konfigurasi produksi	`check --deploy` di CI; DEBUG default mati	Aktifkan CSP bertahap (mode report-only dulu)
E2E	Belum ada	Playwright: login+2FA, tambah anggota, persetujuan, ekspor
Performa	Belum ada	k6 untuk daftar anggota & dashboard; cek N+1
Monitoring	Belum ada	Sentry + uptime check + endpoint /healthz
Backup	Belum terdokumentasi	Cron dump terenkripsi + latihan restore tiap kuartal

Perintah harian di SPBD-PLP

```
python manage.py test                      # seluruh suite
python manage.py test config.tests_audit   # hanya audit otorisasi
python manage.py check --deploy            # cek keamanan konfigurasi
bandit -r . -x tests -ll                   # SAST
pip-audit -r requirements.txt              # kerentanan paket
python manage.py migrate --plan            # lihat migrasi yang akan jalan
```

Smoke test minimal untuk SPBD-PLP setelah deploy

- Halaman login 200 dan HTTPS valid.
- Login akun uji (Admin) + 2FA berhasil.
- Dashboard dan peta Highcharts termuat; tidak ada galat konsol.
- Daftar anggota terbuka dan terbatas sesuai wilayah akun uji koordenador.
- Foto anggota hanya terbuka bila login (akses tanpa login harus ditolak).
- Ekspor CSV berjalan dan sel berbahaya sudah dinetralkan.

9. Daftar Alat 2026

Kategori	Alat (gratis / sumber terbuka diutamakan)
Lint / format / tipe	ruff, black, mypy, pyright, ESLint, Prettier, TypeScript
Unit & integrasi	pytest, pytest-django, Django TestCase, factory_boy, Faker, freezegun, Testcontainers
E2E & visual	Playwright (utama), Cypress, Percy
API	schemathesis, Postman/Newman, Bruno, Pact
SAST / rahasia	bandit, semgrep, CodeQL, gitleaks, trufflehog
Dependensi / SBOM	pip-audit, Dependabot, Renovate, OSV-Scanner, Syft, CycloneDX
DAST	OWASP ZAP, Nuclei, Burp Suite Community

Kategori	Alat (gratis / sumber terbuka diutamakan)
Container / IaC	Trivy, Grype, hadolint, Checkov
Beban & performa	k6, Locust, Lighthouse CI, WebPageTest
Aksesibilitas	axe-core, Pa11y, Lighthouse, NVDA / VoiceOver (manual)
Lintas peramban	Playwright, BrowserStack, LambdaTest
Observability	OpenTelemetry, Prometheus, Grafana, Loki, Sentry, GlitchTip
Uptime / sintetis	UptimeRobot, Better Stack, Uptime Kuma (self-host), Grafana Synthetic
Mutasi	mutmut, Stryker, cosmic-ray
Chaos / ketahanan	Chaos Mesh, Toxiproxy (tingkat lanjut)

> Alat berganti cepat; prinsipnya tetap. Pilih sedikit alat yang **benar-benar dipakai** dan masuk pipeline, daripada banyak alat yang tak dijalankan.

10. Checklist Go / No-Go

Sebelum deploy

- [ ] Semua test (unit, integrasi, otorisasi, E2E) hijau di CI
- [ ] Tidak ada temuan keamanan tinggi/kritis terbuka (SAST, dependensi, ZAP)
- [ ] `check --deploy` tanpa peringatan; DEBUG mati; SECRET_KEY dari environment
- [ ] Migrasi diuji pada salinan data; `makemigrations --check` bersih
- [ ] Backup produksi baru dibuat dan restore pernah diuji
- [ ] Performa diuji (Core Web Vitals + beban) dan lolos ambang
- [ ] Aksesibilitas dasar dan tampilan ponsel diperiksa
- [ ] Rencana rollback tertulis; catatan rilis siap; waktu rilis tidak di jam puncak / Jumat sore

Saat deploy

- [ ] Tag versi dibuat; artefak yang sama dipakai staging & produksi
- [ ] Staging lulus smoke test
- [ ] Strategi rilis (canary/blue-green/jendela perawatan) ditetapkan
- [ ] Orang yang bertugas siaga dan dasbor terbuka

Setelah deploy

- [ ] Smoke test produksi lulus
- [ ] Error rate & latensi normal 30 menit pertama
- [ ] Alert dan uptime check aktif untuk versi baru
- [ ] Tinjauan 24 jam dan 1 minggu dijadwalkan
- [ ] Feature flag/jalur lama dibersihkan; dokumentasi diperbarui
- [ ] Bila ada insiden: post-mortem + test regresi ditambahkan

11. Kesalahan Umum dan Jalur Belajar

Kesalahan yang paling sering

- Mengejar angka coverage tanpa menguji aturan penting (otorisasi, validasi).
- Terlalu banyak E2E rapuh, terlalu sedikit integrasi.
- Staging berbeda jauh dari produksi sehingga 'di staging jalan, di produksi rusak'.
- Menguji hanya jalur bahagia; tidak menguji input buruk dan pengguna tidak berhak.
- Backup ada tetapi tidak pernah diuji pulih.
- Tidak ada monitoring: baru tahu rusak dari keluhan pengguna.
- Rahasia (password, kunci) tertinggal di repo atau log.
- Deploy Jumat sore tanpa rencana rollback.

Jalur belajar 8 minggu

Minggu	Fokus	Latihan
1	Dasar testing, pytest/Django test	Tulis 10 unit test untuk fungsi utilitas proyek
2	Integrasi & database	Uji alur tambah anggota + validasi form
3	Otorisasi & keamanan dasar	Matriks peran x URL; uji XSS, CSRF, IDOR
4	CI/CD	Pasang pipeline contoh bab 7 di repositori
5	Playwright E2E	3 skenario kritis + trace saat gagal
6	Performa & aksesibilitas	Skrip k6 + Lighthouse CI + audit keyboard
7	Deploy aman	Latihan rilis ke staging, smoke test, rollback
8	Observability & insiden	Pasang Sentry + uptime; simulasikan insiden dan tulis post-mortem

Sumber rujukan

- OWASP: Top 10, ASVS, Testing Guide (WSTG), Cheat Sheet Series - owasp.org
- Dokumentasi resmi: Django (Security, Testing, Deployment checklist), Playwright, k6, pytest
- Web.dev: Core Web Vitals; W3C WCAG 2.2
- Buku: 'Google SRE Book' (gratis daring), 'Accelerate' (praktik CI/CD)

> Satu kalimat untuk diingat: **test cepat dan banyak di awal, gerbang ketat saat rilis, pengamatan terus-menerus sesudahnya.**